

NYCU DLP Lab4

i. Derivate conditional VAE formula

$$\log p(x; \theta) = \int q_{\phi}(z|x) \log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} dz + \text{KL} (q_{\phi}(z|x) \| p(z; \theta))$$

In VAE, maximize the log likelihood of the data x .

- decoder 之 parameter 為 θ , encoder 為 ϕ

$$\log p(x; \theta) = \int q_{\phi}(z|x) \log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} dz + \text{KL} (q_{\phi}(z|x) \| p(z; \theta))$$

In CVAE , 觀測 c 是條件變量

$$\log p(x|c; \theta) = \int q_{\phi}(z|x, c) \log \frac{p_{\theta}(x, z|c)}{q_{\phi}(z|x, c)} dz + \text{KL} (q_{\phi}(z|x, c; \phi) \| p(z|c; \theta)) \geq \text{Lb}$$

$$\because p(x, z) = p(z)p(x|z)$$

$$\log p(x|c; \theta) = \int q_{\phi}(z|x, c) \log \frac{p_{\theta}(x|z, c)p(z|c; \theta)}{q_{\phi}(z|x, c; \phi)} dz$$

所以我們要最大化ELBO:

$$L(x, c; \theta, \phi) = \int q_{\phi}(z|x, c; \phi) \log \frac{p_{\theta}(x|z, c)p(z|c; \theta)}{q_{\phi}(z|x, c; \phi)} dz$$

$$= -\text{KL}(q_{\phi}(z|x, c; \phi) || p(z|c; \theta)) + \mathbb{E}_{q_{\phi}(z|x, c; \phi)} \log p_{\theta}(x|z, c; \theta)$$

- KL divergence: $-\text{KL}(q_{\phi}(z|x, c; \phi) || p(z|c; \theta))$
- Reconstruction loss: $\mathbb{E}_{q_{\phi}(z|x, c; \phi)} \log p_{\theta}(x|z, c; \theta)$

In VAE, maximize the log likelihood of the data x
 其中 decoder 的 parameter 為 θ , encoder 為 ϕ

$\log p(x; \theta) = \int q(z|x, \phi) \log \frac{p(x, z; \theta)}{q(z|x, \phi)} dz + \text{KL}(q(z|x, \phi) || p(z|x; \theta))$

In cVAE, 其中 c 是條件變量

$\Rightarrow \log p(x, c; \theta) = \int q(z|x, c; \phi) \log \frac{p(x, z, c; \theta)}{q(z|x, c; \phi)} dz + \text{KL}(q(z|x, c; \phi) || p(z|x, c; \theta)) \geq L_b$

$\therefore$ we want to maximize the **ELBO**

$L(x, c; \theta, \phi) = \int q(z|x, c; \phi) \log \frac{p(x|z, c; \theta) p(z|c; \theta)}{q(z|x, c; \phi)} dz$

$= \int q(z|x, c; \phi) \log \frac{p(x|z, c; \theta)}{q(z|x, c; \phi)} + \int q(z|x, c; \phi) \log p(z|c; \theta) dz$

$= -\text{KL}(q(z|x, c; \phi) || p(z|c; \theta)) + \mathbb{E}_{q(z|x, c; \phi)} \log p(x|z, c; \theta)$

KL divergence reconstruction loss

ii. Introduction

In this assignment, it will train a Variational Autoencoder (VAE) for video prediction. The process involves encoding the input's first frame and the skeleton image, then using a Gaussian Predictor to predict μ and $\log \sigma$ and performing reparameterization. Subsequently, the Generator will produce the next frame of the video. This step is repeated to complete the prediction.

Additionally, we implement the teacher-forcing method, and KL-annealing to observe the predicting result.

2. Lab Description

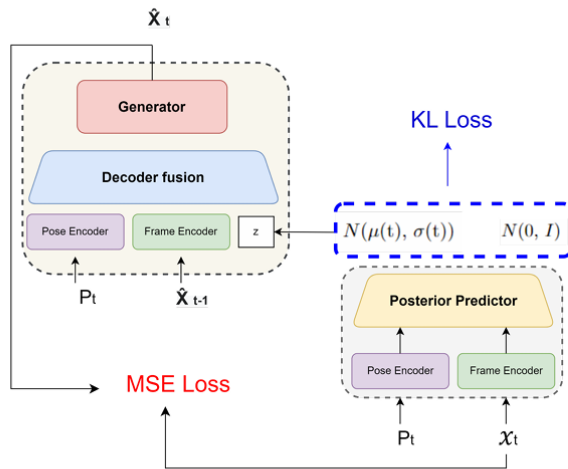


Figure 2. (a)

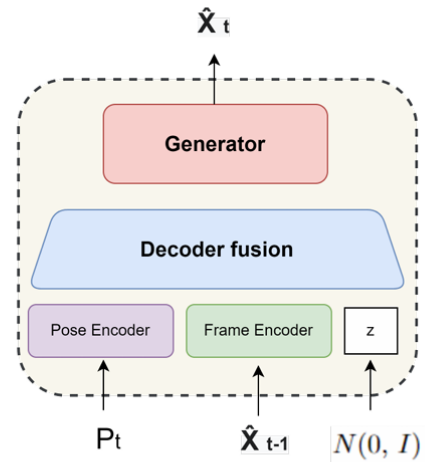


Figure 2. (b)

iii. Implementation details

1. How do you write your training/testing protocol

- **First:** Permute the dimensions so that we can split the time series from zero to `train_vi_len-1`.
- **Second:** Initialize the values, including the loss and output image.
- **Third:** Transform the image from the RGB domain to the feature domain and pass it to the Gaussian Predictor to get z , μ , and $\log\text{var}$.
- **Fourth:** Check whether to use the teacher-forcing method.
- **Fifth:** Use the Decoder Fusion to obtain the parameters, which will be passed to the Generator to produce the result.
- **Sixth:** Calculate the loss, which includes reconstruction loss and KL divergence. We use β to control the ratio of KL divergence.
- **Seventh:** Compute gradients and update the weights.

```

def training_one_step(self, img, label, adapt_TeacherForcing): #[batch,時間序列,channels,height,width]
    #permute 成 [時間序列,batch,channels,height,width]

    img=img.permute(1,0,2,3,4)
    label = label.permute(1,0,2,3,4)

    #Define initital value
    KL=0
    Reconstruction_loss=0
    pred = img[0]
    for i in range(self.train_vi_len-1): #處理dataset裡面的一個data 因為包含多個時間序列 (16)

        #transform image from RGB-domain to feature-domain
        img_out=self.frame_transformation(img[i]) #img_out [batch,c,h,w]
        label_out=self.label_transformation(label[i])

        z,mu,logvar = self.Gaussian_Predictor(img_out,label_out) # 是用這一偵來產生 distribution
        KL += kl_criterion(mu, logvar, batch_size = self.batch_size)
        if i>=1:
            if adapt_TeacherForcing: #用前一偵的結果
                img_last = img[i-1]
            else:
                img_last = pred
            img_last_out=self.frame_transformation(img_last)
            #pass to decoder
            param=self.Decoder_Fusion(img_last_out,label_out,z) #根據上一偵產生結果
            pred = self.Generator(param)
            #calculate loss
            Reconstruction_loss += self.mse_criterion(pred, img[i])

    loss = Reconstruction_loss + self.kl_annealing.get_beta() * KL

    self.optim.zero_grad()
    loss.backward()

    self.optimizer_step()

    return loss.detach()

```

- In the testing phase, I sample z from a normal distribution and pass it to the decoder to generate the corresponding image."

```

def val_one_step(self, img, label, idx=0):
    img = img.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
    label = label.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
    assert label.shape[0] == 630, "Testing pose sequence should be 630"
    assert img.shape[0] == 1, "Testing video sequence should be 1"

    # decoded_frame_list is used to store the predicted frame seq
    # label_list is used to store the label seq
    # Both list will be used to make gif
    decoded_frame_list = [img[0].cpu()]
    label_list = []
    PSNR_total=0
    pred=img[0]
    for i in range(1,self.val_vi_len): #處理dataset裡面的一個data 因為包含多個時間序列 (630)

        img_out=self.frame_transformation(pred)
        label_out=self.label_transformation(label[i])

        #pass encoder

        z = torch.cuda.FloatTensor(1, self.args.N_dim, self.args.frame_H, self.args.frame_W).normal_()#從normal distribution smaple一個出來

        param=self.Decoder_Fusion(img_out,label_out,z)
        pred = self.Generator(param)

        decoded_frame_list.append(pred.cpu())
        label_list.append(label[i].cpu())

```

2.How do you implement reparameterization tricks

The purpose of the reparameterization method is to sample z from the posterior distribution using the reparameterization trick, allowing gradients to propagate through the random variable. Here, we generate a random variable ϵ from a normal distribution, and then use the mean

μ and standard deviation σ to calculate z .

$$\epsilon \sim \mathcal{N}(0, I) \quad \sigma = \exp\left(\frac{\log(\text{var})}{2}\right) \quad z = \mu + \sigma \cdot \epsilon$$

```

#VAE參數化是分布是不可微分的
#重參數化的技巧是假設 prior distribution 是 Normal Gaussian ~
#，因此可以將 latent space 以
#表示，則使得分布與網路參數分布無關，使其可以計算 Gradient。
def reparameterize(self, mu, logvar): #直接從後驗分佈採z比較困難 所以使用reparameterize
    #透過encoder 得到 mu 和 log var 使用 reparameterize 得到隨機變量z (要是normal distribution)
    # TODO
    # 生成大小mu 的 normal distribution
    epsilon = torch.randn_like(mu)

    # 算標準差
    std = torch.exp(0.5 * logvar)
    z = mu + std * epsilon
    return z
#raise NotImplementedError

```

3.How do you set your teacher forcing strategy

- Goal: In the initial stage, we tend to use the original image to stabilize the training results. In the late stage, we hope to use the predicted results to help the model learn the correct things.

```
def teacher_forcing_ratio_update(self):
    # TODO
    #tfr_sde:The epoch that teacher forcing ratio start to decay
    #tfr_d_step : Decay step that teacher forcing ratio adopted (0.1)
    if self.current_epoch >= self.tfr_sde:
        # start decay
        self.tfr-=self.tfr_d_step
        self.tfr=max(0,self.tfr)

    #raise NotImplementedError
```

4.How do you set your kl annealing ratio

- We can use the Monotonic and Cyclic to choose the strategy.
- Goal: In VAE training, in order to maximize the reconstruction loss and minimize the KL-divergence, it is common to encounter the KL-vanishing problem, where the latent distribution becomes unrelated to the input. To address this issue, we use KL-annealing to prevent this phenomenon.

```

#KL (Kullback-Leibler) annealing
# 在 VAE 訓練的過程中，為了最大化 reconstruction term 並最小化 KL-divergence term，
# 很容易造成 KL-vanishing (posterior collapse) 的問題，這會使得 latent space distribution
# 直接與輸入 x 無關(因為最小化 KL-divergence 只需使得 posterior = prior)，也這會使得我們的 decoder
# 能力放大而不需仰賴 latent space distribution。因此為解決這項問題，需要使用 KL-annealing
# 循環或線性的調整 KL-divergence term 的係數，保證模型有多一點的時間學習輸入 x 的資訊到 latent space distribution 中。
#lower bound = reconstruction loss + KL (q(z|x)||p(z))
#L=Reconstruction Loss+β·KL Divergence
#KL annealing 的過程可以理解為逐步增大β的值，使得模型在訓練初期更注重Reconstruction loss，
class kl_annealing():
    def __init__(self, args, current_epoch=0):
        self.args=args
        self.current_epoch=current_epoch
        self.type = args.kl_anneal_type #linear step cyclic
        self.cycle = args.kl_anneal_cycle #變化週期
        self.ratio = args.kl_anneal_ratio # 變化的大小 >1 需要更多時間到達最終值
        self.total_epoch = args.num_epoch
        if self.type== 'None':
            self.beta=1.0
        else: #Monotonic cyclic
            self.beta=0.0
        #raise NotImplementedError

    def update(self):
        # TODO
        self.current_epoch+=1
        if self.type !='None':
            self.frange_cycle_linear(n_iter = self.current_epoch,n_cycle=self.cycle , ratio=1)

    def get_beta(self):
        return self.beta

    def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0, n_cycle=10, ratio=1):

        step = 0
        n_pos= n_iter % n_cycle
        step = (n_pos/n_cycle)*ratio #mapping到0~1 代表step
        if(step <= stop):
            self.beta = step
        else: #設為1
            self.beta = stop

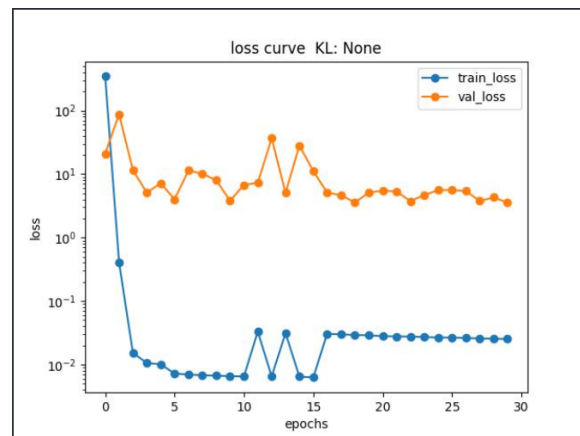
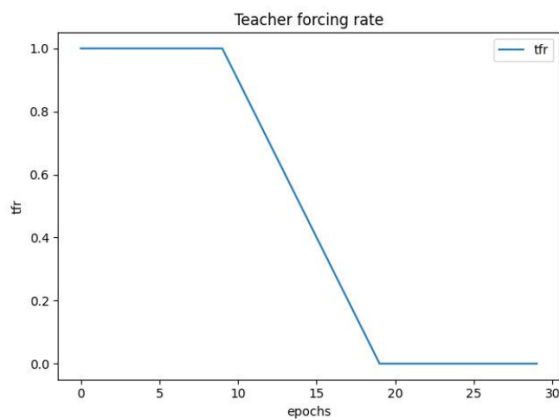
        if(self.type=='Monotonic' and n_iter>=n_cycle): #若monotonic 後期一直設為1
            self.beta=stop

```

iv. Analysis & Discussion

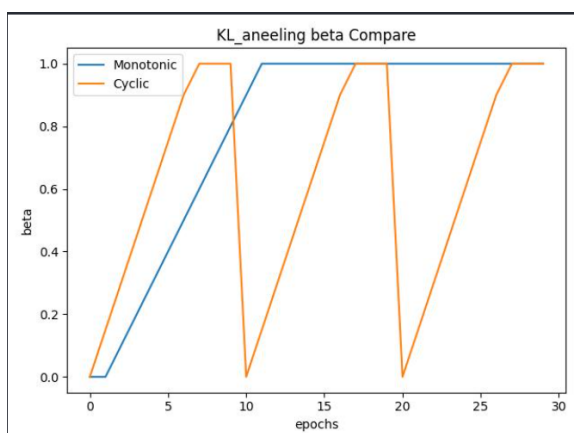
1. Plot Teacher forcing ratio

a. Analysis & compare with the loss curve



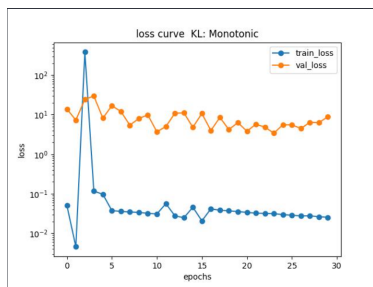
- We can see that the teacher forcing rate starts to decay at epoch 10. Meanwhile, the loss curve increases suddenly. This is because the input may switch to the predicted image, causing the loss to increase abruptly.
- After several rounds of testing, I discovered that setting `tfr` to zero might yield better results. This could be because introducing some imperfections into the model allows it to learn more effectively.

2. Plot the loss curve while training with different settings. Analyze the difference between them

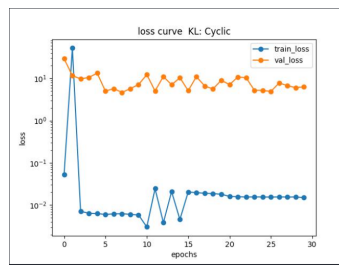


- The beta rate, we implement two kind of KL-annealing method, including the Monotonic and Cyclic.
- I set the cycle for ten epochs

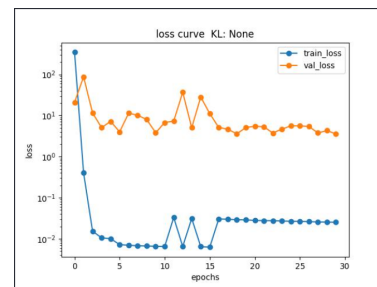
- Compare to three method



**a. With KL annealing
(Monotonic)**



**b. With KL annealing
(Cyclical)**



c. Without KL annealing

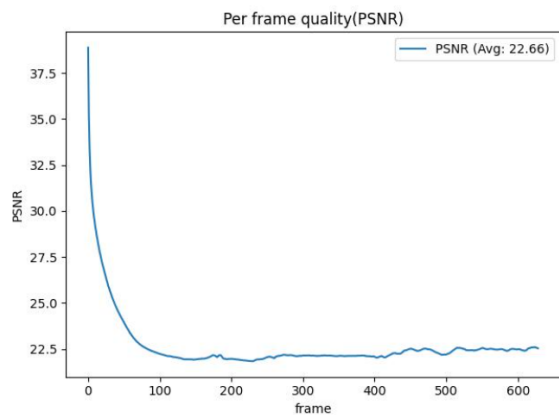
- At epoch 3, the loss suddenly increase, which might be due to a sudden increase in the KL-loss component, causing the model's loss to spike unexpectedly.
- With Cyclical learning rates, at epoch 10, the training loss suddenly decreases because the beta value suddenly drops to 0, leading to continued stable learning in subsequent epochs.
- Compared to Monotonic learning rates, the loss performance with Cyclical learning rates appears to be more unstable. Because the Monotonic beta is stable at 1.
- Compared to not using the KL-annealing method, its loss curve does not show any significant fluctuations. This is because the beta rate is always 1, consistently balancing the KL-divergence weight.



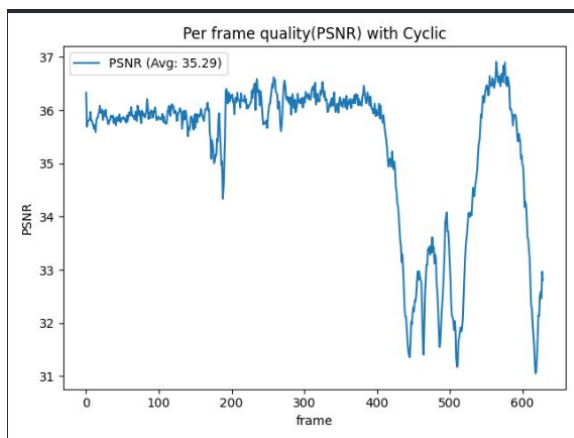
Summarize

- **With KL annealing (Monotonic):**
 - At epoch 3, the loss suddenly increases, likely due to a sudden spike in the KL-loss component, causing the model's loss to unexpectedly rise.
 - The beta value increases gradually during training, leading to stable but slightly fluctuating training loss reduction.
- **With KL annealing (Cyclical):**
 - At epoch 10, the training loss suddenly decreases because the beta value drops to 0, leading to continued stable learning in subsequent epochs.
 - Compared to monotonic annealing, the loss performance with cyclic learning rates is more unstable as the beta oscillates between 0 and 1.
- **Without KL annealing:**
 - The loss curve shows no significant fluctuations because the beta value is always 1, consistently balancing the KL divergence weight.
 - It did not perform a dramatic change at epoch 3 like the other two methods.

3. Plot the PSNR-per frame diagram in validation dataset



- In the initial validation, at the beginning of the video, the PSNR value is high, indicating that the model is making accurate predictions. However, as the video progresses, the model's predictive performance gradually deteriorates, resulting in lower PSNR values later on.



- In the later validation, we can see that the PSNR rate improves, but between frames 450 and 500, it appears to be lower. This could be because the movement in the video is too big, making it difficult for the model to predict accurately.

4. Use the SGD optimizer to convergence the result

- After training with Adam 50 epochs, I use the SGD optimizer to convergence the result
- SGD is more likely to find a global optimal solution in the later stages of training because it introduces more randomness during convergence, which helps to escape local optima.
- SGD's smaller update steps are more likely to find model parameters that generalize better.
- Through the loss curve and PSNR graph we can see that the SGD optimizer give us the convergence to better result

